🤖 Claude Code 实战全指南：从工具到神级队友

一、 核心心法：心态转变 (Mindset Shift)

不要把 Claude Code 当成聊天机器人，而要把它当成你的“代理式编码者” (Agentic Coder) 或“同事”。

- **不仅是输入提示词：** 如果只把它当 Prompt 工具，你只发挥了它 10% 的价值。
- **管理心态：** 你不再是写代码的工人，而是一个管理者。你的任务是设定高阶目标、提供上下文，并验证最终用户体验。

二、 第一级：打好基础 (Foundation)

1. 项目的“大脑”：`claude.md`

这是 Claude Code 的核心记忆文件，决定了它对项目的理解深度。

- **操作：** 在项目根目录下创建 `claude.md`。
- **内容：** 写入项目的高层规则、整体架构、最佳实践、甚至是团队的 Git 工作流规范。

  claude.md文件是项目的“内存”，Claude Code执行任务时会将其添加到上下文中。它包含规则、架构概述、构建命令、测试和调试最佳实践等，确保Claude遵循团队约定，避免误推主分支。
- **进阶技巧：**
  - **快速生成：** 直接使用 `/init` 命令让 Claude 帮其生成初始版本。
  - **目录隔离：** 大项目可以在不同子文件夹放置独立的 `claude.md` 以精确上下文。
  - **自动更新：** 可以要求 Claude 在完成任务后自己去更新这个文件。
  - **外部引用：** 在文件中引用外部文档链接。

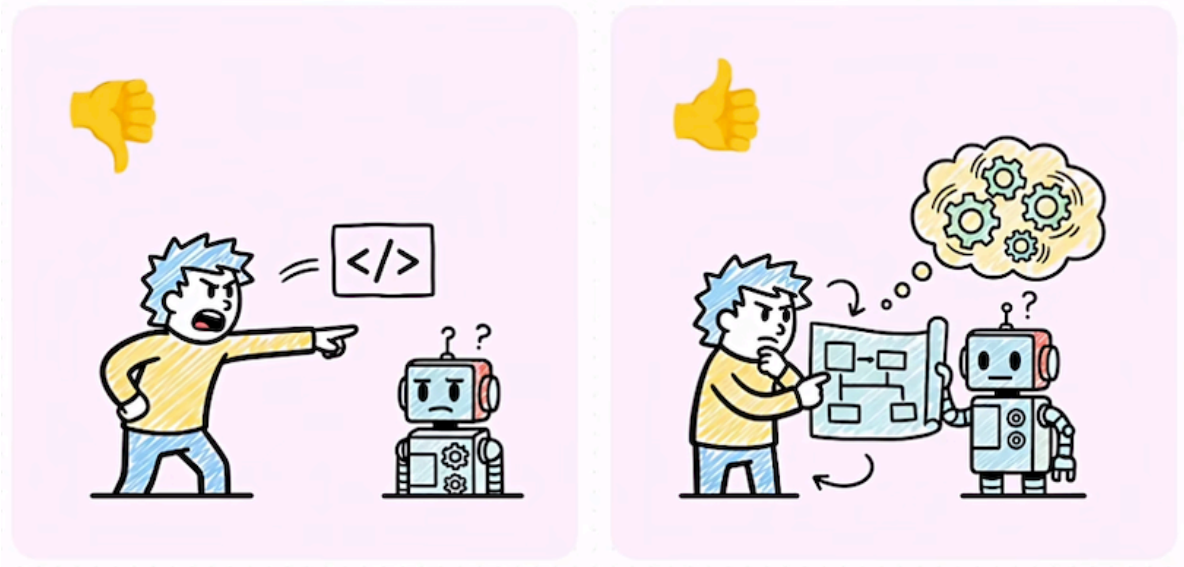
```
### Design Principles
- Comprehensive design checklist in `/context/design-principles.md`
- S-tier SaaS dashboard standards (inspired by Stripe, Airbnb, Linear)
- Focus on user experience, performance, and accessibility (WCAG AA+)    "WC
- Brand style guide in `/context/style-guide.md`
- When making visual (front-end, UI/UX) changes, always refer to `/context/
design-principles.md` and `/context/style-guide.md` for guidance on design
principles and Snapbar branding
```

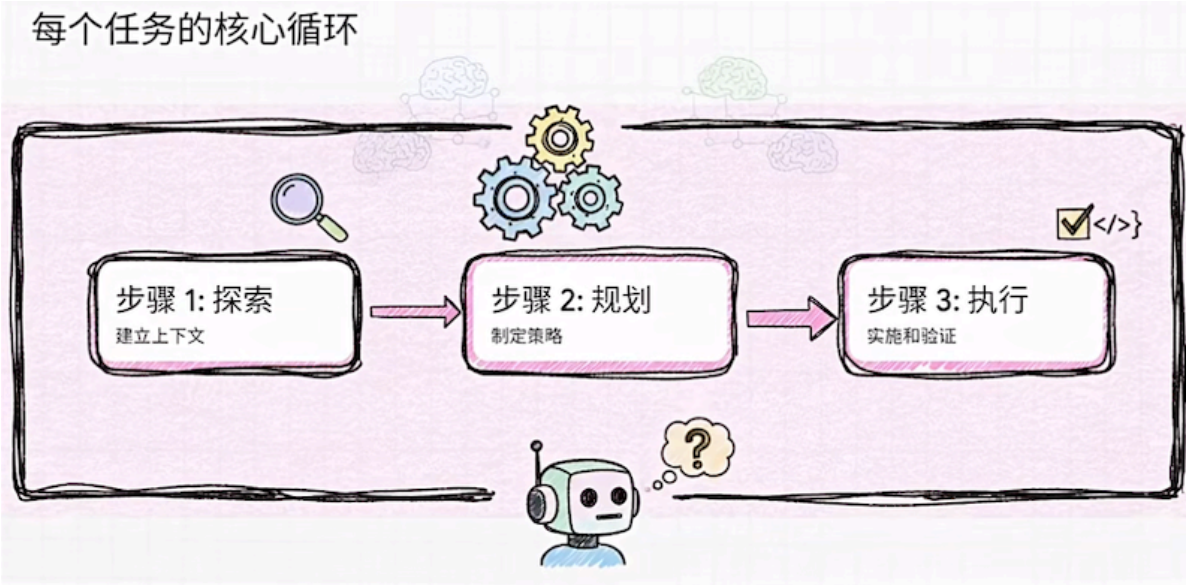
2. 操作神技：双击 `Escape` 键

- **单击 Esc：** 立即喊停（打断胡说八道）。
- **双击 Esc：** **“读档”功能**。回到之前的某个对话点，开辟全新的对话分支。
- **价值：** 节省 Token 开销，避免无效上下文堆积，省钱又省时。

三、 第二级：金牌工作流 (Workflow)

黄金三步法：探索 -> 规划 -> 执行



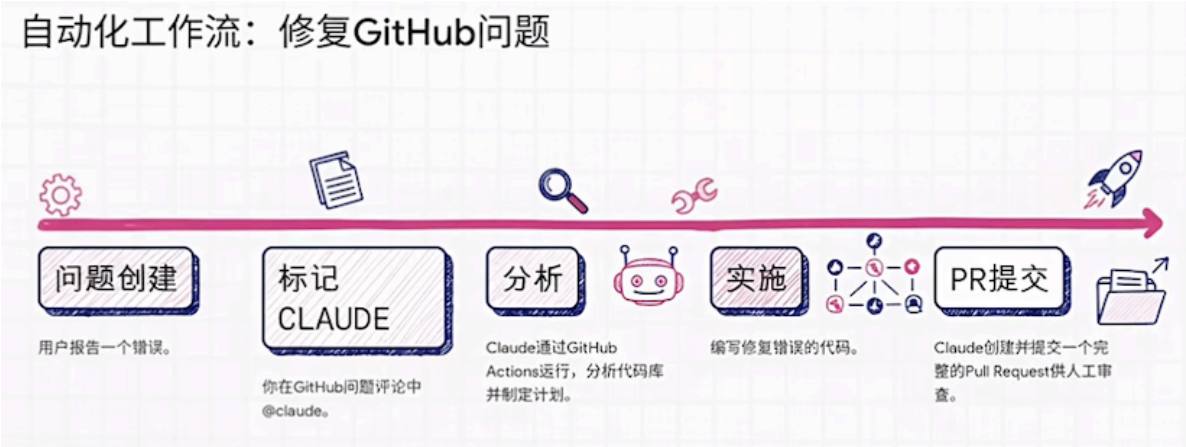


不要一上来就让它写代码，严格遵守以下流程：

- 让 Claude 花时间熟悉代码库、研究 API 如何工作、建立完整的上下文。
- **规划 (Plan)：** 在动手前先出蓝图（逻辑方案）。
- **执行 (Execute)：** 根据确认后的方案生成高质量代码。

实战场景：全自动 Bug 修复

在 GitHub Issue 评论区 `@claude` -> 它会自动分析代码 -> 制定计划 -> 编写代码 -> 提交 Pull Request (PR)。全程无需人工干预。



四、 第三级：大师进阶技巧 (Master Class)

1. 开发者技巧：强迫它“反思”

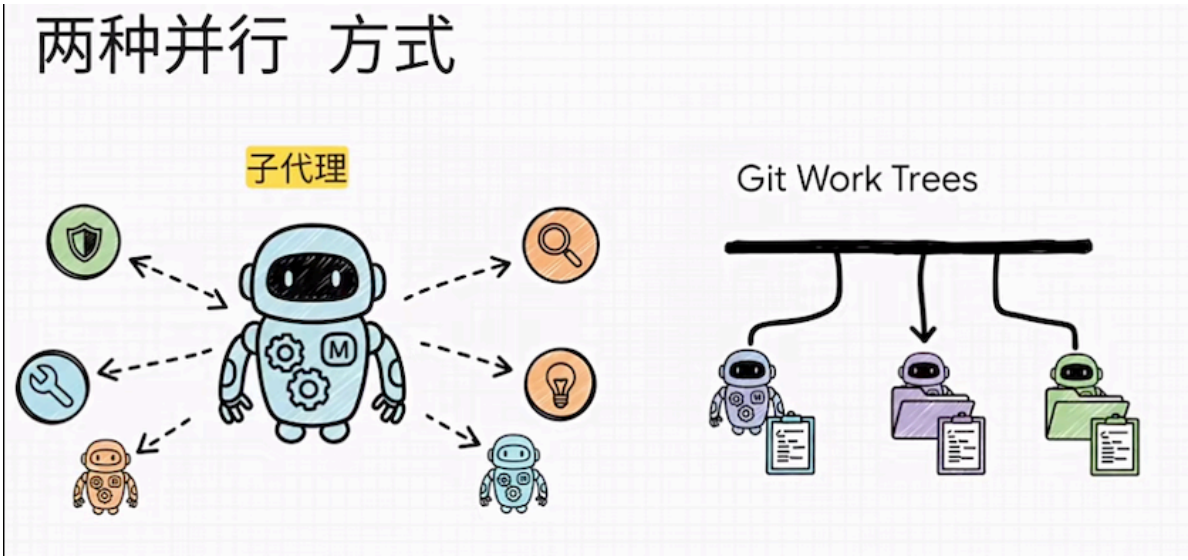
为了防止 Claude 变成只会说“计划通”的“复读机”，可以使用以下技巧：

- **话术示例：** “我的另一个开发者提出了这个计划 [附上计划]，你觉得怎么样？”
- **效果：** 此时 Claude 会从“跟班模式”切换为“严厉评审员模式”，给出更具批判性和客观性的反馈，帮你挖出隐藏的坑。

2. 并行处理 (Parallel Processing)

摆脱传统的“单打独斗”模式，开启“脑暴小组”模式。

方式	描述
子代理 (Sub-agents)	将大任务拆解，派给不同的专家代理（如：安全审查员、测试编写员），同时开工。
Git Worktrees	在同一个项目里开辟多个独立工作区，让几个 Claude 实例用不同的方法解决同一个问题，你最后当裁判选优。
代理蜂群 (Swarms)	极端情况下，可启动成百上千个 AI 代理互相协作或竞争。



五、 总结：新旧工作方式对比

维度	过去（提示词工程师）	现在（管理者/代理模式）
交互方式	一行行给指令，盯着干活	设定高阶目标，提供背景
关注点	抠实现细节，逐行审代码	验证用户体验，把控全局
角色定义	微观管理者	团队负责人 (Lead)
核心工具	复杂的 Prompt	`claude.md` + 代理流

旧方式 (提示者)	新方式 (管理者)
编写逐行提示	提供高阶目标和上下文
微观管理代码	验证最终输出和体验
对抗工具限制	管理AI代理团队

> **关键金句：** 放弃逐行审查代码的旧习惯，在更高的抽象层次验证输出。你未来的价值不再是写出“美的代码”，而是控局。